

PAGE DE GARDE

Projet Parking Métropole Groupe B

Woittin Kilyan & Moreau Mark

Langage C

SOMMAIRE

Page de Garde1

1) Introduction 3

2) Fonctionnalités Implémentées 4

3) Difficultés Rencontrées et Solutions 7

4) Commentaires et Suggestions 8

5) Auto-évaluation 9

6) Lien GitHub et Structure du Code 10

1) Introduction

Ce projet a pour but la gestion d'un réseau de parking avec l'utilisation d'un fichier csv, Par le biais de plusieurs fonctions permettant la gestion des entrées et sorties et l'actualisation des données des parkings, il a été réalisé sur Visual Studio Code

Technologies: C (standard + Windows.h pour Sleep), CSV, Visual Studio Code

2) Fonctionnalités Implémentées

1. Chargement des parkings depuis un fichier CSV

Fonction : lesparkings()

Description : Lecture du fichier parking-metropole (2).csv et stockage dans un tableau de structures.

Choix technique : Utilisation de fgets et strtok pour parser efficacement les données.

2. Affichage des parkings

Fonctions : afficheParking() et afficheParkings()

Description : Affichage d'un parking spécifique ou de tous les parkings chargés.

Choix : Lecture sélective pour améliorer la lisibilité.

3. Gestion de l'entrée d'un véhicule

Fonction : entrerParking()

Description : Enregistrement de l'heure d'entrée et de l'immatriculation du client.

4. Gestion de la sortie d'un véhicule

Fonction : sortieParking()

Description : Calcul du montant à payer selon la durée de stationnement (simulée avec Sleep lors des tests).

Détail technique : Utilisation de time_t pour la gestion du temps.

5. Mise à jour du nombre de places disponibles

Fonction : mettreAJourOccupation()

Description : Incrémentation/décrémentation du nombre de places après entrée ou sortie.

6. Système d'authentification administrateur

Fonction : modeAdministrateur()

Description : Protection de certaines opérations via un mot de passe.

g. Sauvegarde de l'état des parkings

Fonction : sauvegarderEtatParking()

Description : Enregistrement de l'état actuel des parkings dans un nouveau fichier CSV.

```

struct Parking{
char identifiant [100];
char Nom[100];
char Adresse[100];
char Ville[100];
char Etat[100];
int Place_disponible;
int capacite_max;
char date_de_mise_a_jour[100];
char affichage_panneaux[100];
};

struct Parking liste[100];

```

Céation de la structure des Parkings

C'est grâce à cette structure que l'on peut manipuler les Parkings et les triée quand nous les utilisons.

```

int lesparkings(){
    // Ouverture du fichier source
    FILE *fichier = fopen("parking-metropole (2).csv","r");
    if (fichier == NULL) {
        printf("Erreur d'ouverture. ");
        return -1;
    }
    char ligne[1000];
    int i=0;
    fgets(ligne, sizeof(ligne), fichier);
    while (fgets(ligne, sizeof(ligne), fichier)) { // Lire une ligne
        //printf("%s", ligne);
        const char * separators = ","; //separations des tokens
        // define strToken
        strcpy(liste[i].identifiant, strtok ( ligne, separators ));
        strcpy(liste[i].Nom , strtok ( NULL, separators ));
        strcpy(liste[i].Adresse , strtok ( NULL, separators ));
        strcpy(liste[i].Ville , strtok ( NULL, separators ));
        strcpy(liste[i].Etat , strtok ( NULL, separators ));
        liste[i].Place_disponible = atoi(strtok ( NULL, separators ));
        liste[i].capacite_max = atoi(strtok ( NULL, separators ));
        strcpy(liste[i].date_de_mise_a_jour , strtok ( NULL, separators ));
        strcpy(liste[i].affichage_panneaux , strtok ( NULL, separators ));
        i++;
    }

    printf("il y a %i parking possible",i);
    fclose(fichier);
    return i;
}

```

La fonction parkings permet d'utiliser le fichier csv pour inscrire grâce aux tokens les différentes informations dans la structure Parkings, nous avons eu des soucis de type a causé par les type string qui sont naturellement résolut avec strcpy.

```
✓ struct Client{  
    char Immatriculation [100];  
    int Montant_paye;  
    time_t heure_entrer;  
    time_t heure_sorti;  
};  
  
struct Client listeInfo[1000];
```

Pour faire un suivi des clients nous avons décidé d'utiliser une autre structure client plutôt qu'un autre fichier cvs car dans notre cas cela semble plus logique car on connaît le nombre max de clients possible qui est égale a la somme des capaciter des différents parkings en soit on sait combien de place louer dans l'espace mémoire de la structure, on note notemment l'utilisations du type time_t car il est nécessaires pour contrôler les heures d'entrer et de sorti

3) Difficultés Rencontrées et Solutions

Parsing du fichier CSV : Difficulté à gérer les lignes avec strtok au début.

Problème des types : Au commencement difficulté a géré la différence des types et donc à leurs besoins spécifiques en soit l'utilisation de strcpy strcmp

Gestion du temps sous Windows : time_t fonctionnait mais pour les tests rapides, nous avons ajouté Sleep pour simuler l'attente.

Compréhension : la compréhension de ce qui était demandé des fonctions entrer et sortie n'était pas claire à cause des problème des mise à jour à effectuer sur les informations des parkings durant un enregistrement de client l'actualisation des donnée n'est pas faite dans l'ordre exacte c'est à dire que le les fonctions entrer et sortie sont correct mais ne suivent pas exactement les exécutions demander par exemple l'heure d'entrer est inscrit dans la fonction entrer pas dans la fonction miseAjouroccupation car on ne pouvait pas faire un return de plusieurs variables ce qui a pour but d'éviter cette problématique en se débarrassent du surplus .

Mode Administrateur : trouver une façon au mode administrateur résolu en cherchant un peu mais pas instinctif dès le début à cause du scanf que on avait très peu utilisé

4) Commentaires et Suggestions

Points positifs : La gestion des structures et des fichiers est devenue plus intuitive au fil du projet.

Améliorations possibles : Implémenter un système de menus pour l'utilisateur final. Ajouter une interface graphique simple (type Curses ou même une petite appli Windows).

Apprentissages : Ça nous a appris à manipuler des fichiers, gérer les structures et simuler des scénarios réels.

5) Auto-évaluation

Planification : 7/10 (quelques ajustements en cours de route à la cause de Mark qui n'avait pas les horaires de tp de disponibles)

Codage : 7/10 (code globalement propre, totalement subjectif je trouve donc pas plus il manque de la visibilité avec les commentaires on n'a pas mis beaucoup)

Tests : 6/10 (tests fonctionnels répond aux demandes les fonctions ne bug pas si on suit les procédure test plus approfondis à faire car pas très pousser je trouve)

Amélioration future : Plus de clarté dans les commentaires de code, plus de test poussé et des fonctions dynamique serait sympa.

6) Lien GitHub

Lien GitHub : <https://github.com/Tigreri/Projet-CGroupB>

Fin du Compte-Rendu